

Week 1: Basics of Python

❖ Day 1: Introduction, Setup, Variables, Input/Output

References

- W3Schools - Python Intro (https://www.w3schools.com/python/python_intro.asp)
- Programiz - Python Basics (<https://www.programiz.com/python-programming>)
- GeeksforGeeks - Python Basics (<https://www.geeksforgeeks.org/python-programming-language/>)
- JavaTpoint - Python Tutorial (<https://www.javatpoint.com/python-tutorial>)
- YouTube: [CodeWithHarry – Python for Beginners \(Full Course\)](#)

Problems

1. Print "Hello, World!"
2. Store and print your name, age, and city.
3. Take two numbers as input and print their sum.
4. Write a program to swap two variables.
5. Print type of variables: int, float, str.
6. Take user input and print with a message.
7. Concatenate two strings.
8. Print multiple variables in one line.
9. Check type of input (use type()).
10. Print a formatted string using f-string.

❖ Day 2: Data Types & Type Casting

References

- W3Schools - Data Types (https://www.w3schools.com/python/python_datatypes.asp)
- Programiz - Data Types (<https://www.programiz.com/python-programming/variables-datatypes>)
- GeeksforGeeks - Python Data Types (<https://www.geeksforgeeks.org/python-data-types/>)
- JavaTpoint - Data Types (<https://www.javatpoint.com/python-data-type>)
- YouTube: [Tech With Tim – Python Variables & Data Types](#)

Problems

1. Assign int, float, string, list, dict and print type.
2. Convert float → int, int → str.
3. Convert string number "45" to int.
4. Convert list to tuple.
5. Check type of True, None, 3+4j.
6. Add int and float → show result type.
7. Convert ASCII to character using chr().
8. Convert character to ASCII using ord().
9. Take input and cast into float.
10. Use isinstance() for type checking.

❖ Day 3: Operators

References

- W3Schools – Operators (https://www.w3schools.com/python/python_operators.asp)
- Programiz – Operators (<https://www.programiz.com/python-programming/operators>)
- GeeksforGeeks -Python Operators (<https://www.geeksforgeeks.org/python-operators/>)
- JavaTpoint – Operators (<https://www.javatpoint.com/python-operators>)
- YouTube: [CodeWithHarry – Operators in Python](#)

Problems

1. Add, subtract, multiply, divide two numbers.
2. Floor divide and modulus operation.
3. Find power using **.
4. Compare two numbers using relational operators.
5. Use logical operators with and, or, not.
6. Use bitwise AND, OR, XOR.
7. Left shift a number by 2.
8. Right shift a number by 3.
9. Use assignment operators (+=, *=).
10. Evaluate an expression (a+b) * c - d.

❖ Day 4: Strings & String Methods

References

- W3Schools – Strings (https://www.w3schools.com/python/python_strings.asp)
- Programiz – Strings (<https://www.programiz.com/python-programming/string>)
- GeeksforGeeks - Python Strings (<https://www.geeksforgeeks.org/python-strings/>)
- JavaTpoint – Strings (<https://www.javatpoint.com/python-string>)
- YouTube: [Tech With Tim – Strings in Python](#)

Problems

1. Print length of a string.
2. Convert string to upper and lower case.
3. Check if string is digit/alphabet.
4. Slice a string ([2:6]).
5. Reverse a string using slicing.
6. Replace a word in string.
7. Count frequency of a character.
8. Split string into words.
9. Join list of strings into one.
10. Check if substring exists.

❖ Day 5: Conditional Statements

References

- W3Schools - If Else (https://www.w3schools.com/python/python_conditions.asp)
- Programiz - If Statement (<https://www.programiz.com/python-programming/if-elif>)
- GeeksforGeeks - Conditional Statements (<https://www.geeksforgeeks.org/python-if-else/>)
- JavaTpoint - If Else (<https://www.javatpoint.com/python-if-else>)

- YouTube: [CodeWithHarry – If-Else Explained](#)

Problems

1. Check if a number is positive or negative.
2. Check if number is even or odd.
3. Find largest of 3 numbers.
4. Grade student based on marks.
5. Check if year is leap year.
6. Check voting eligibility (age ≥ 18).
7. Check if number divisible by 3 and 5.
8. Nested if: student pass/fail with grade.
9. Check character is vowel or consonant.
10. Use ternary operator for comparison.

❖ Day 6: Loops

References

- W3Schools – Loops (https://www.w3schools.com/python/python_for_loops.asp)
- Programiz – Loops (<https://www.programiz.com/python-programming/for-loop>)
- GeeksforGeeks – Loops (<https://www.geeksforgeeks.org/loops-in-python/>)
- JavaTpoint – Loops (<https://www.javatpoint.com/python-loops>)
- YouTube: [Tech With Tim – Loops in Python](#)

Problems

1. Print numbers from 1 to 10.
2. Print even numbers from 1 to 20.
3. Sum of first 50 numbers.
4. Factorial of a number.
5. Multiplication table of a number.
6. Reverse a number using while loop.
7. Count digits in a number.
8. Print Fibonacci sequence.
9. Nested loop: print pattern (triangle).
10. Break & continue demo.

Week 2: Data Structures in Python

❖ Day 1: Lists, List Methods

Topics

- Creating lists, accessing elements, slicing
- Built-in list methods (append, extend, insert, remove, pop, sort, reverse, count, index)
- Nested lists and list operations

References

- Programiz – Python Lists (<https://www.programiz.com/python-programming/list>)
- GeeksforGeeks – Python List (<https://www.geeksforgeeks.org/python-list/>)
- JavaTpoint – Python Lists (<https://www.javatpoint.com/python-lists>)
- W3Schools – Python Lists (https://www.w3schools.com/python/python_lists.asp)
- YouTube: [Telusko – Python Lists](#)

Problems

1. Create a list of 10 numbers and print the **first, last, and middle elements**.
2. Write a program to **reverse a list** without using slicing.
3. Add 5 elements to a list using **append()** and **extend()**.
4. Remove all **duplicate elements** from a list.
5. Find the **maximum and minimum** elements in a list.
6. Sort a list in **ascending and descending order**.
7. Count the **frequency of each element** in a list.
8. Merge two lists into a single list.
9. Write a program to **rotate a list** by k positions.
10. Find the **second largest** element in a list.

❖ Day 2: Tuples & Sets

Topics

- Creating tuples, immutability, tuple methods (count, index)
- Creating sets, set methods (add, remove, discard, union, intersection, difference, issubset, issuperset)
- Frozen sets

References

- Programiz – Python Tuple (<https://www.programiz.com/python-programming/tuple>)
- GeeksforGeeks – Python Tuples (<https://www.geeksforgeeks.org/python-tuples/>)
- JavaTpoint – Python Tuples (<https://www.javatpoint.com/python-tuples>)
- W3Schools – Python Tuples (https://www.w3schools.com/python/python_tuples.asp)
- Programiz – Python Set (<https://www.programiz.com/python-programming/set>)
- GeeksforGeeks – Python Sets (<https://www.geeksforgeeks.org/python-sets/>)
- JavaTpoint – Python Sets (<https://www.javatpoint.com/python-sets>)
- W3Schools – Python Sets (https://www.w3schools.com/python/python_sets.asp)
- YouTube: [Corey Schafer – Sets & Tuples in Python](#)

Problems

1. Create a tuple with 5 numbers and print its **length**.
2. Convert a list to a tuple and vice versa.
3. Write a program to **find the index** of an element in a tuple.
4. Unpack a tuple into **separate variables**.
5. Create two sets and perform **union, intersection, and difference**.
6. Remove duplicates from a list using **set()**.
7. Check whether a set is a **subset/superset** of another.
8. Find the **symmetric difference** of two sets.
9. Create a **frozen set** and try modifying it.
10. Write a program to check whether all elements of one set are present in another.

❖ **Day 3: Dictionaries**Topics

- Creating dictionaries, key-value pairs
- Dictionary methods (get, keys, values, items, update, pop, popitem, clear)
- Nested dictionaries, dictionary comprehension

References

- Programiz – Python Dictionary (<https://www.programiz.com/python-programming/dictionary>)
- GeeksforGeeks – Python Dictionary (<https://www.geeksforgeeks.org/python-dictionary/>)
- JavaTpoint – Python Dictionaries (<https://www.javatpoint.com/python-dictionary>)
- W3Schools – Python Dictionaries (https://www.w3schools.com/python/python_dictionaries.asp)
- YouTube: [Corey Schafer – Python Dictionaries](#)

Problems

1. Create a dictionary of **5 students with marks** and print it.
2. Access the value of a key safely using **get()**.
3. Update a key-value pair in a dictionary.
4. Merge two dictionaries.
5. Remove a key-value pair using **pop()**.
6. Count the frequency of characters in a string using a dictionary.
7. Write a program to **invert a dictionary** (swap keys and values).
8. Find the **key with the maximum value** in a dictionary.
9. Check if a key exists in a dictionary.
10. Write a program to **sort a dictionary by values**.

❖ **Day 4: Comprehensions (List/Dict/Set)**Topics

- List comprehensions (with conditions, nested)
- Dictionary comprehensions
- Set comprehensions
- Comparison with loops

References

- Programiz – List Comprehension (<https://www.programiz.com/python-programming/list-comprehension>)
- GeeksforGeeks – Python List Comprehension (<https://www.geeksforgeeks.org/python-list-comprehension/>)
- JavaTpoint – List Comprehension (<https://www.javatpoint.com/python-list-comprehension>)
- W3Schools – Python List Comprehension (https://www.w3schools.com/python/python_lists_comprehension.asp)
- YouTube: [Programming with Mosh – Python List Comprehensions](#)

Problems

1. Create a list of **squares of numbers** from 1 to 10.
2. Create a list of **even numbers** from 1 to 50.
3. Extract vowels from a string using list comprehension.
4. Flatten a 2D list using list comprehension.
5. Generate a dictionary of numbers and their squares.
6. Create a dictionary mapping each character in a string to its frequency.
7. Create a set of all **unique characters** in a string.
8. Filter out odd numbers from a list using comprehension.
9. Generate a list of tuples (x, x²) for numbers 1 to 10.
10. Write a comprehension to extract **words longer than 4 letters** from a sentence.

❖ **Day 5: Functions (def, return, recursion)**Topics

- Defining and calling functions
- Function parameters (positional, keyword, default, variable-length)
- Return statement
- Recursion basics with examples (factorial, Fibonacci)

References

- Programiz – Python Functions (<https://www.programiz.com/python-programming/function>)
- GeeksforGeeks – Python Functions (<https://www.geeksforgeeks.org/python-functions/>)
- JavaTpoint – Python Functions (<https://www.javatpoint.com/python-functions>)
- W3Schools – Python Functions (https://www.w3schools.com/python/python_functions.asp)
- YouTube: [Telusko – Python Functions](#)

Problems

1. Write a function to check whether a number is **prime**.
2. Write a function to return the **factorial** of a number (recursion).
3. Write a function to check if a string is a **palindrome**.
4. Write a function to calculate the **Fibonacci sequence** (recursion).
5. Write a function that takes a list and returns the **sum of elements**.
6. Write a function that returns the **maximum of three numbers**.
7. Write a function to find the **GCD** of two numbers.

8. Write a function to count the number of vowels in a string.
9. Write a recursive function to **reverse a string**.
10. Write a recursive function to calculate the **sum of digits** of a number.

❖ Day 6: Lambda, map, filter, reduce

Topics

- Anonymous functions with lambda
- map(), filter(), reduce() (with functools)
- Practical use cases

References

- Programiz – Lambda Function (<https://www.programiz.com/python-programming/anonymous-function>)
- GeeksforGeeks – Lambda Functions (<https://www.geeksforgeeks.org/python-lambda-anonymous-functions-filter-map-reduce/>)
- JavaTpoint – Python Lambda (<https://www.javatpoint.com/python-lambda-functions>)
- W3Schools – Lambda (https://www.w3schools.com/python/python_lambda.asp)
- YouTube: [Corey Schafer – Python Lambda, Map, Filter, Reduce](#)

Problems

1. Use a lambda function to square a number.
2. Use map() to convert a list of strings to uppercase.
3. Use map() to get the cube of numbers from 1 to 10.
4. Use filter() to extract even numbers from a list.
5. Use filter() to extract words starting with 'a' from a list.
6. Use reduce() to find the **sum of a list**.
7. Use reduce() to find the **maximum element** in a list.
8. Write a program using lambda to **sort a list of tuples by second element**.
9. Write a lambda expression to check if a number is **divisible by 3**.
10. Write a program using map() and lambda to add two lists element-wise.

Week 3: Advanced Python Programming

❖ Day 1: File Handling (read/write)

Topics

- Opening and closing files (open(), with open())
- File modes (r, w, a, rb, wb)
- Reading methods: read(), readline(), readlines()
- Writing with write() and writelines()
- Handling file paths
- File operations in text vs binary

References

- Programiz → Python File Handling (<https://www.programiz.com/python-programming/file-operation>)
- GeeksforGeeks → File Handling in Python (<https://www.geeksforgeeks.org/file-handling-python/>)
- JavaTpoint → Python File Handling (<https://www.javatpoint.com/python-file-handling>)
- W3Schools → Python File Handling (https://www.w3schools.com/python/python_file_handling.asp)
- **YouTube:** Telusko – [Python File Handling Tutorial](#)

Problems

1. Create a text file and write 10 lines of text.
2. Read the file and print all lines.
3. Count the number of words in a file.
4. Count the number of lines in a file.
5. Append a new line to an existing file.
6. Copy contents from one file to another.
7. Replace a specific word in a file.
8. Read a file and print only lines containing a certain word.
9. Write a program to merge multiple files into one.
10. Read a CSV file and print the sum of numbers in a column.

❖ Day 2: Exception Handling

Topics

- Errors vs Exceptions
- try, except, finally
- else in exceptions
- Catching multiple exceptions
- Raising exceptions (raise)
- Custom exceptions

References

- Programiz → Python Exceptions (<https://www.programiz.com/python-programming/exception-handling>)
- GeeksforGeeks → Python Exception Handling (<https://www.geeksforgeeks.org/python-exception-handling/>)

- JavaTpoint → Python Exception Handling (<https://www.javatpoint.com/python-exception-handling>)
- W3Schools → Python Try Except (https://www.w3schools.com/python/python_try_except.asp)
- **YouTube:** Programming with Mosh – [Python Error Handling](#)

Problems

1. Handle **division by zero** exception.
2. Handle **file not found** exception.
3. Handle **type errors** when adding a string and a number.
4. Use else and finally blocks in a try-except.
5. Raise a **ValueError** if input is negative.
6. Catch multiple exceptions in one try block.
7. Write a program that handles **IndexError** for list access.
8. Handle exception for converting a string to int.
9. Implement a **custom exception** for invalid age input.
10. Write a program to retry reading a file if IOError occurs.

❖ Day 3: Modules & Packages (math, random, sys)

Topics

- Creating and importing modules (import, from import)
- Built-in modules: math, random, sys
- Exploring module functions
- Packages and __init__.py
- Installing external packages with pip

References

- Programiz → Python Modules (<https://www.programiz.com/python-programming/modules>)
- GeeksforGeeks → Modules in Python (<https://www.geeksforgeeks.org/python-modules/>)
- JavaTpoint → Python Modules (<https://www.javatpoint.com/python-modules>)
- W3Schools → Python Modules (https://www.w3schools.com/python/python_modules.asp)
- **YouTube:** Telusko – [Python Modules & Packages](#)

Problems

1. Use math to calculate factorial and power.
2. Generate 10 random numbers between 1–100.
3. Pick a random element from a list.
4. Shuffle a list using random.shuffle().
5. Use sys.argv to read command-line arguments.
6. Write a custom module with a function and import it.
7. Use math.gcd() to find the GCD of two numbers.
8. Calculate square root of a number using math.sqrt().
9. Use random.sample() to pick multiple elements.
10. Print system path using sys.path.

❖ Day 4: Iterators & Generators

Topics

- Iterator protocol: `__iter__()` and `__next__()`
- `iter()` and `next()` functions
- Generator functions with `yield`
- Generator expressions
- Difference between iterators & generators

References

- Programiz → Python Iterators (<https://www.programiz.com/python-programming/iterator>) | Python Generators (<https://www.programiz.com/python-programming/generator>)
- GeeksforGeeks → Iterators (<https://www.geeksforgeeks.org/iterators-in-python/>) | Generators (<https://www.geeksforgeeks.org/generators-in-python/>)
- JavaTpoint → Iterators (<https://www.javatpoint.com/python-iterator>) | Generators (https://www.w3schools.com/python/python_generators.asp)
- W3Schools → Python Iterators (https://www.w3schools.com/python/python_iterators.asp) | Python Generators (https://www.w3schools.com/python/python_generators.asp)
- **YouTube:** Corey Schafer – [Python Generators Explained](#)

Problems

1. Create an iterator for a list and print elements using `next()`.
2. Use a for loop to traverse an iterator.
3. Create a generator to yield squares of numbers 1–10.
4. Create a generator to yield even numbers up to 50.
5. Implement Fibonacci using a generator.
6. Create a generator to read large files line by line.
7. Use `iter()` with a sentinel value.
8. Chain two iterators using `itertools.chain`.
9. Write a generator that yields prime numbers up to 100.
10. Write a generator for infinite sequence of numbers divisible by 5.

❖ Day 5: Decorators & Context Managers

Topics

- Functions as first-class objects
- Introduction to decorators
- Writing custom decorators
- Using `@decorator` syntax
- Context Managers (with statement)
- Creating custom context managers with `__enter__` and `__exit__`

References

- Programiz → Python Decorators (<https://www.programiz.com/python-programming/decorator>) | Python Context Managers (<https://www.programiz.com/python-programming/context-manager>)
- GeeksforGeeks → Decorators (<https://www.geeksforgeeks.org/decorators-in-python/>) | Context Managers (<https://www.geeksforgeeks.org/context-manager-in-python/>)

- JavaTpoint → Python Decorators (<https://www.javatpoint.com/python-decorator>) | Context Manager (<https://www.javatpoint.com/python-context-manager>)
- W3Schools → Python Decorators (*no direct context manager page, but decorators are covered*) (https://www.w3schools.com/python/python_iterators.asp)
- **YouTube:** Corey Schafer – [Python Decorators Tutorial](#)

Problems

1. Write a decorator to **print "Function started" and "Function ended"**.
2. Write a decorator to **measure execution time** of a function.
3. Implement multiple decorators on a single function.
4. Use with to open and read a file.
5. Create a context manager class using `__enter__` and `__exit__`.
6. Use with statement to write to a file.
7. Write a decorator to **log function arguments**.
8. Create a decorator to **check login credentials** before function execution.
9. Implement a context manager to temporarily change a directory.
10. Write a decorator that **caches results** of a function.

❖ Day 6: Regular Expressions

Topics

- Introduction to re module
- Meta-characters (`.`, `^`, `$`, `*`, `+`, `?`, `{}`, `[]`, `\`, `|`, `()`)
- Functions: `match()`, `search()`, `findall()`, `sub()`, `split()`
- Raw strings in regex (`r"""`)
- Practical regex use cases

References

- Programiz → Python RegEx (<https://www.programiz.com/python-programming/regex>)
- GeeksforGeeks → Python RegEx (<https://www.geeksforgeeks.org/python-regex/>)
- JavaTpoint → Python RegEx (<https://www.javatpoint.com/python-regex>)
- W3Schools → Python RegEx (https://www.w3schools.com/python/python_regex.asp)
- **YouTube:** Corey Schafer – [Python RegEx Tutorial](#)

Problems

1. Check if a string starts with a vowel.
2. Check if a string ends with a digit.
3. Find all email addresses in a text.
4. Replace all spaces with underscores.
5. Extract all numbers from a string.
6. Validate a phone number using regex.
7. Split a text into words using regex.
8. Check for repeated words in a string.
9. Remove all non-alphanumeric characters from a string.
10. Extract all dates from a text in format DD/MM/YYYY.

Week 4: Object-Oriented Programming in Python

❖ Day 1: Classes & Objects

Topics

- Introduction to OOP in Python
- Creating a class using class
- Creating objects (instances of a class)
- Instance variables vs class variables
- Instance methods vs class methods vs static methods
- `__str__()` method

References

- Programiz: Python Classes and Objects (<https://www.programiz.com/python-programming/class>)
- GeeksforGeeks: Python Classes and Objects (<https://www.geeksforgeeks.org/python-classes-and-objects/>)
- JavaTpoint: Python OOPs Concepts (<https://www.javatpoint.com/python-oops-concepts>)
- W3Schools: Python Classes/Objects (https://www.w3schools.com/python/python_classes.asp)
- YouTube: [Corey Schafer - Python OOP Tutorial 1: Classes and Instances](#)

Problems

1. **Employee**: attributes name, basic_pay; methods to compute HRA, DA, gross, net (with tax).
2. **BankAccount**: deposit(), withdraw(), get_balance(); reject overdraft.
3. **Student**: store 5 marks, total(), percentage(), grade(), `__str__()` formatted card.
4. **Rectangle**: area(), perimeter(), and a compare(other) that returns larger by area.
5. **Point2D**: distance(other) using Euclidean distance; midpoint(other).
6. **Complex** (custom): add, subtract, multiply; `__str__()` as a + bi.
7. **Book**: apply_discount(percent); ensure price never negative.
8. **Temperature**: static methods c_to_f(), f_to_c(); demo both.
9. **Counter**: class var counting instances; @classmethod get_count().
10. **InventoryItem**: attributes sku, name, price, qty; methods restock(n), sell(n) with validation.

❖ Day 2: Constructors & Destructors

Topics

- The `__init__()` constructor
- Default vs parameterized constructors
- Initializing attributes
- Destructor `__del__()` in Python
- Garbage collection and object lifecycle

References

- Programiz: Python Constructors (<https://www.programiz.com/python-programming/constructor>)

- GeeksforGeeks: Constructors in Python (<https://www.geeksforgeeks.org/constructors-in-python/>)
- JavaTpoint: Constructors in Python (<https://www.javatpoint.com/constructor-in-python>)
- W3Schools: Python Classes Example (https://www.w3schools.com/python/python_classes.asp) (includes constructor usage)
- YouTube: [Telusko - Constructors in Python](#)

Problems

1. **Student:** auto-increment roll_no via class var in `__init__`; `__str__()` to show details.
2. **Rational:** constructor takes num, den; reduce to simplest form; error on den=0.
3. **Time:** constructor accepts "HH:MM" string; validate and store h, m; to `_24h()`.
4. **Vector:** default zero vector if no args; alternative constructor from `_list(cls, lst)`.
5. **Config:** constructor loads key-values from dict; `get(key, default)`; show default args.
6. **Logger:** open a file in `__init__`, write messages; ensure resource is released (show `__del__` conceptually).
7. **Account:** optional min_balance param with default; validate on creation.
8. **Rectangle:** allow construction from width, height OR side (square) via `@classmethod square(side)`.
9. **Person:** constructor with name, age; raise `ValueError` if age < 0; `__repr__()` for debugging.
10. **DatabaseConnection (mock):** flags connected in `__init__`; `close()`; destructor prints "closed" (for lifecycle demo).

❖ Day 3: Inheritance (Single, Multiple, Multilevel)

Topics

- Concept of inheritance
- Single inheritance
- Multiple inheritance
- Multilevel inheritance
- `super()` function
- Method Resolution Order (MRO)

References

- Programiz: Python Inheritance (<https://www.programiz.com/python-programming/inheritance>)
- GeeksforGeeks: Inheritance in Python (<https://www.geeksforgeeks.org/inheritance-in-python/>)
- JavaTpoint: Inheritance in Python (<https://www.javatpoint.com/inheritance-in-python>)
- W3Schools: Python Inheritance (https://www.w3schools.com/python/python_inheritance.asp)
- YouTube: [Corey Schafer - Python OOP Tutorial 3: Inheritance](#)

Problems

1. **Shape** → **Circle/Rectangle:** base `area()` placeholder; derived classes implement actual area.
2. **Vehicle** → **Car/Bike:** use `super()` to set brand, model; add wheels in subclasses.

3. **Person** → **Employee** → **Manager**: multilevel; each adds fields; details() uses super() chain.
4. **BankAccount** → **SavingsAccount**: override withdraw() to block if balance < min_balance.
5. **Device** → **Phone/Tablet**: override specs(); demonstrate polymorphic call on list of devices.
6. **Multiple inheritance**: Loggable mixin adds log(); Timestamped mixin adds created_at; class Task(Loggable, Timestamped).
7. **Diamond problem**: A → B, C → D; print MRO and show super() resolving the chain.
8. **FileHandler** → **CsvHandler/JsonHandler**: override read() to parse accordingly (mock data).
9. **Animal** → **Dog/Cat**: override sound(); iterate through list calling sound().
10. **Calculator** → **ScientificCalculator**: add power(), sin(), cos(); reuse base operations with super().

❖ Day 4: Polymorphism & Method Overriding

Topics

- Understanding polymorphism
- Method overriding in Python
- Function overloading vs method overloading
- Duck typing in Python
- Operator overloading (__add__, __len__, etc.)

References

- Programiz: Polymorphism in Python (<https://www.programiz.com/python-programming/polymorphism>)
- GeeksforGeeks: Polymorphism in Python (<https://www.geeksforgeeks.org/polymorphism-in-python/>)
- JavaTpoint: Polymorphism in Python (<https://www.javatpoint.com/polymorphism-in-python>)
- W3Schools: Python Polymorphism (https://www.w3schools.com/python/python_polymorphism.asp)
- YouTube: [Amulya's Academy - Polymorphism in Python](#)

Problems

1. **speak() polymorphism**: classes Human, Dog, Robot each implement speak(); iterate and call.
2. **Operator overloading**: Vector2D with __add__, __sub__, __eq__, __repr__.
3. **Length semantics**: Playlist class implementing __len__ (number of songs) and __iter__.
4. **Comparable objects**: Employee with __lt__ to sort by salary then name; sort a list.
5. **Duck typing**: function draw(obj) that calls obj.draw() for Circle and Square classes.
6. **Method overriding**: Employee.calculate_bonus() overridden in Engineer, Manager.
7. **Callable objects**: class Counter with __call__ to increment and return current count.
8. **Contextual polymorphism**: format_price(x) works for int, float, and Money class with __float__.

9. **File exporters:** classes PDFExporter, CSVExporter both implement export(data); pass either at runtime.
10. **Resource wrapper:** SafeList overriding __getitem__ to handle out-of-range gracefully (returns None).

❖ Day 5: Abstraction & Encapsulation

Topics

- Concept of abstraction
- Abstract classes using abc module
- Encapsulation in Python
- Private (__var) and protected (_var) members
- Getters and setters (@property, @setter)

References

- Programiz: Python OOP Concepts (<https://www.programiz.com/python-programming/object-oriented-programming>)
- GeeksforGeeks: Abstraction in Python (<https://www.geeksforgeeks.org/abstract-classes-in-python/>)
- JavaTpoint: Abstraction in Python (<https://www.javatpoint.com/abstraction-in-python>)
- W3Schools: Python Classes Example (https://www.w3schools.com/python/python_classes.asp)
- YouTube: [Telusko - Abstraction and Encapsulation in Python](#)

Problems

1. **Payment (ABC):** abstract pay(amount); concrete CreditCardPayment, UPIPayment.
2. **Shape (ABC):** abstract area(), perimeter(); implement Rectangle, Triangle.
3. **Account (encapsulation):** private __balance; deposit, withdraw with checks; balance read-only property.
4. **Temperature (properties):** private __celsius; @property celsius, @celsius.setter validates range.
5. **BankCard:** hide __pin; method verify_pin(pin); prevent direct access.
6. **Inventory:** private __items dict; add_item(), remove_item() controlled via methods only.
7. **User:** email property validates regex on set; raises ValueError if invalid.
8. **Document (ABC):** abstract render(); PDFDocument, HTMLDocument implement differently.
9. **RateLimiter:** encapsulate request counts; expose allow(); keep internals hidden.
10. **Student:** private marks list; property returns a copy (to prevent external mutation).

❖ Day 6: OOPs Mini-Project (Student Management System)

Topics

- Apply OOP concepts to build a mini project
- Class: Student (attributes: name, roll, marks)
- Class: StudentManager (methods: add student, remove student, search student, display all students)
- Demonstrate inheritance, encapsulation, and polymorphism

References

- Programiz: Python OOP Example (<https://www.programiz.com/python-programming/object-oriented-programming>)
- GeeksforGeeks: Python Project Ideas (<https://www.geeksforgeeks.org/python-projects-beginner-to-advanced/>)
- JavaTpoint: Python Projects (<https://www.javatpoint.com/python-projects>)
- W3Schools: Python Classes Example (https://www.w3schools.com/python/python_classes.asp)
- YouTube: [CodeWithHarry - Python OOP Project](#)

Feature Tasks

1. **Model classes:** Student(id, name, branch, marks: dict) with total(), percentage(), grade().
2. **Manager:** StudentManager with add_student(), remove_student(id), get_student(id).
3. **Search:** search by name (partial), branch, and by min_percentage.
4. **Update:** update_marks(id, subject, marks) with validation (0–100).
5. **Reports:** print toppers per subject and overall topper.
6. **Persistence:** save/load students to/from CSV (file I/O + error handling).
7. **Encapsulation:** keep marks private; use properties/methods for updates and reads.
8. **Inheritance:** add ScholarStudent(Student) with extra scholarship_amount and overridden __str__.
9. **Polymorphism:** export(formatter) where formatter can be CSVFormatter or JSONFormatter (both have format(list_of_students) method).
10. **CLI menu:** looped menu to perform CRUD, search, reports; handle invalid input with exceptions.

Week 5: Python Libraries & Data Handling

❖ Day 1: NumPy Basics

Concepts

- Introduction to NumPy.
- Installing NumPy (pip install numpy).
- Creating arrays (1D, 2D, multi-dimensional).
- Checking data type, shape, size.
- Basic properties of arrays.

Problems

1. Install NumPy and check its version.
2. Create a 1D NumPy array with values from 0 to 9.
3. Create a 3×3 NumPy array filled with all zeros.
4. Create a 3×3 NumPy array filled with all ones.
5. Create a 3×3 identity matrix.
6. Generate an array of even numbers from 10 to 50.
7. Create a 1D array of 10 random numbers between 0 and 1.
8. Find the shape, size, and datatype of an array.
9. Convert a Python list [1,2,3,4,5] into a NumPy array.
10. Create a 5×5 matrix with values 1 to 25 and reshape it into (25,1).

References

- Programiz – NumPy Tutorial (<https://www.programiz.com/python-programming/numpy>)
- GeeksforGeeks – NumPy Basics (<https://www.geeksforgeeks.org/python-numpy/>)
- JavaTpoint – NumPy Introduction (<https://www.javatpoint.com/numpy-tutorial>)
- W3Schools – NumPy Tutorial (https://www.w3schools.com/python/numpy_intro.asp)
- YouTube: [NumPy Full Course – ProgrammingKnowledge](#)

❖ Day 2: NumPy Arrays & Operations

Concepts

- Array indexing and slicing.
- Mathematical operations (add, subtract, multiply, divide).
- Aggregate functions (sum, mean, std, max, min).
- Broadcasting.

Problems

1. Create a 1D array of 10 numbers and print the 2nd to 6th elements.
2. Extract every alternate element from a 1D array.
3. Perform element-wise addition of two 1D arrays.
4. Multiply two 2D arrays (element-wise).
5. Find the mean, median, and standard deviation of an array.
6. Create a 4×4 matrix and find the sum of each row.
7. Create a 3×3 matrix and find the max in each column.
8. Use broadcasting to add [1,2,3] to a 3×3 matrix.
9. Reverse a NumPy array.
10. Flatten a multi-dimensional array into 1D.

References

- Programiz – NumPy Array Operations (<https://www.programiz.com/python-programming/numpy/array>)
- GeeksforGeeks – NumPy Operations (<https://www.geeksforgeeks.org/numpy-arithmetic-operations/>)
- JavaTpoint – NumPy Array (<https://www.javatpoint.com/numpy-array>)
- W3Schools – NumPy Array Operations (https://www.w3schools.com/python/numpy_array.asp)
- YouTube: [NumPy Arrays & Operations – Corey Schafer](#)

❖ **Day 3: Pandas Basics**Concepts

- Introduction to Pandas.
- Series and DataFrame.
- Creating DataFrame from lists, dicts, and NumPy arrays.
- Reading CSV/Excel files.

Problems

1. Install Pandas and check version.
2. Create a Pandas Series from a Python list [10,20,30,40,50].
3. Create a DataFrame from a dictionary of lists.
4. Create a DataFrame with custom row indices.
5. Load a CSV file into a DataFrame.
6. Print the first 5 and last 5 rows of a DataFrame.
7. Display the column names and data types in a DataFrame.
8. Create a Series with custom index ['a','b','c'].
9. Convert a NumPy array into a DataFrame.
10. Save a DataFrame into a CSV file.

References

- Programiz – Pandas Tutorial (<https://www.programiz.com/python-programming/pandas>)
- GeeksforGeeks – Pandas Basics (<https://www.geeksforgeeks.org/python-pandas-dataframe/>)
- JavaTpoint – Pandas Introduction (<https://www.javatpoint.com/python-pandas>)
- W3Schools – Pandas Tutorial (<https://www.w3schools.com/python/pandas/default.asp>)
- YouTube: [Pandas Tutorial – Krish Naik](#)

❖ **Day 4: Pandas Operations (Series, DataFrame)**Concepts

- Indexing & slicing in Series/DataFrames.
- Selection (loc, iloc).
- Filtering rows & columns.
- Adding/removing columns.
- Handling missing values.

Problems

1. Select the first column of a DataFrame.
2. Select a row using .loc and .iloc.
3. Filter rows where column Age > 25.
4. Add a new column Grade to DataFrame.
5. Drop a column from DataFrame.
6. Handle missing values using fillna.
7. Handle missing values using dropna.
8. Rename columns in a DataFrame.
9. Sort DataFrame by a column.
10. Group DataFrame by a column and compute mean.

References

- Programiz – Pandas Operations (<https://www.programiz.com/python-programming/pandas/dataframe>)
- GeeksforGeeks – Pandas Operations (<https://www.geeksforgeeks.org/python-pandas-dataframe/>)
- JavaTpoint – Pandas Operations (<https://www.javatpoint.com/python-pandas-operations>)
- W3Schools – Pandas Operations (https://www.w3schools.com/python/pandas/pandas_dataframes.asp)
- YouTube: [Pandas Full Course – Data School](#)

❖ **Day 5: Matplotlib Basics (Line, Bar, Scatter Plots)**Concepts

- Introduction to Matplotlib.
- Line plot, Bar plot, Scatter plot.
- Adding labels, title, legend.
- Customizing plots (color, marker, linestyle).

Problems

1. Plot a line graph for $y = 2x + 5$.
2. Plot a line graph with labels and title.
3. Plot a bar chart for student marks.
4. Plot a horizontal bar chart.
5. Plot a scatter plot of random points.
6. Add grid lines to a plot.
7. Customize line color and style.
8. Plot multiple lines on the same graph.
9. Save a plot as png file.
10. Plot a sine wave using NumPy + Matplotlib.

References

- Programiz – Matplotlib (<https://www.programiz.com/python-programming/matplotlib>)
- GeeksforGeeks – Matplotlib Tutorial (<https://www.geeksforgeeks.org/matplotlib-tutorial/>)
- JavaTpoint – Matplotlib (<https://www.javatpoint.com/matplotlib>)
- W3Schools – Matplotlib Tutorial (https://www.w3schools.com/python/matplotlib_intro.asp)

- YouTube: [Matplotlib Full Tutorial – Corey Schafer](#)

❖ Day 6: Seaborn Basics + Visualization Mini Project

Concepts

- Introduction to Seaborn.
- Line plot, Bar plot, Histogram, Heatmap.
- Styling Seaborn plots.
- Mini-project: Data visualization of a real dataset (e.g., Titanic dataset).

Problems

1. Install Seaborn and check version.
2. Load Titanic dataset from Seaborn.
3. Plot distribution of Age column.
4. Create a bar plot of Gender vs Survival.
5. Create a count plot for Passenger class.
6. Create a heatmap for correlation between features.
7. Style plots with darkgrid.
8. Plot multiple plots in one figure.
9. Save Seaborn plot as image.
10. Mini-project: Visualize Titanic dataset with at least 3 different charts.

References

- Programiz – Seaborn Tutorial (<https://www.programiz.com/python-programming/seaborn>)
- GeeksforGeeks – Seaborn Tutorial (<https://www.geeksforgeeks.org/seaborn-tutorial/>)
- JavaTpoint – Seaborn (<https://www.javatpoint.com/seaborn-tutorial>)
- W3Schools – Seaborn (https://www.w3schools.com/python/pandas/pandas_seaborn.asp)
- YouTube: [Seaborn Crash Course – StatQuest](#)

Week 6: Final Projects & Placement Prep

❖ Day 1: Problem-solving using Python (coding contests)

Topics: Competitive coding basics, problem breakdown, time complexity, coding patterns.

Practice Problems

1. Reverse words in a string without using built-in reverse.
2. Find second largest element in a list.
3. Count unique characters in a string.
4. Check if a number is prime.
5. Find missing number in a list (1 to n).
6. Print Pascal's Triangle up to n rows.
7. Rotate an array k steps.
8. Merge two sorted lists.
9. Find longest word in a given sentence.
10. Implement binary search.

References

- Programiz – Python Problems (<https://www.programiz.com/python-programming/examples>)
- GeeksforGeeks – Python Practice (<https://www.geeksforgeeks.org/python-programming-examples/>)
- JavaTpoint – Python Examples (<https://www.javatpoint.com/python-programs>)
- W3Schools – Python Exercises (<https://www.w3schools.com/python/exercise.asp>)
- YouTube: [Coding Interview Python Problems](#)

❖ Day 2: Data Structures Problems (Stacks, Queues)

Topics: Implementation using lists, collections.deque, queue module.

Practice Problems

1. Implement stack using list.
2. Implement queue using list.
3. Balanced parentheses check using stack.
4. Next Greater Element using stack.
5. Reverse a string using stack.
6. Implement circular queue.
7. Evaluate postfix expression using stack.
8. Implement queue using two stacks.
9. Design min-stack (return min element in $O(1)$).
10. Implement priority queue (min-heap).

References

- Programiz –Stack & Queue (<https://www.programiz.com/python-programming/stack>)

- GeeksforGeeks – Stack/Queue Problems (<https://www.geeksforgeeks.org/stack-in-python/>)
- JavaTpoint – Python Data Structures (<https://www.javatpoint.com/python-data-structure>)
- W3Schools – Data Structures (https://www.w3schools.com/python/python_lists.asp)
- YouTube: [Stacks & Queues in Python](#)

❖ Day 3: Searching & Sorting in Python

Topics: Linear search, binary search, bubble/selection/insertion/merge/quick sort.

Practice Problems

1. Implement linear search.
2. Implement binary search.
3. Bubble sort implementation.
4. Selection sort implementation.
5. Insertion sort implementation.
6. Merge sort implementation.
7. Quick sort implementation.
8. Count sort implementation.
9. Sort dictionary by values.
10. Find kth smallest/largest element.

References

- Programiz – Searching & Sorting (<https://www.programiz.com/dsa/algorithm>)
- GeeksforGeeks – Sorting/Searching (<https://www.geeksforgeeks.org/python-searching-and-sorting-algorithms/>)
- JavaTpoint – Sorting in Python (<https://www.javatpoint.com/sorting-algorithms-in-python>)
- W3Schools – Python Sort (https://www.w3schools.com/python/python_lists_sort.asp)
- YouTube: [Sorting Algorithms in Python](#)

❖ Day 4: File-based Mini Project (Banking System / Library Management)

Topics: File handling (open, read/write, append, CSV), data storage, CRUD operations.

Mini-Tasks

1. Create a text file and write data.
2. Append new data to existing file.
3. Read file line by line.
4. Search for a record in file.
5. Count words in a text file.
6. Implement Banking System: Create Account.
7. Deposit & Withdraw (update file).
8. Library Mgmt: Add new book record.
9. Library Mgmt: Search and issue book.
10. Delete record from file.

References

- Programiz – File Handling (<https://www.programiz.com/python-programming/file-operation>)
- GeeksforGeeks – File Handling Projects (<https://www.geeksforgeeks.org/file-handling-python/>)
- JavaTpoint – File Handling (<https://www.javatpoint.com/python-file-handling>)
- W3Schools – File Handling (https://www.w3schools.com/python/python_file_handling.asp)
- YouTube: [Python File Handling Project](#)

❖ Day 5: Data Analysis Project (CSV + Pandas + Visualization)

Topics: Pandas DataFrame, reading CSV, cleaning, Matplotlib/Seaborn plots.

Mini-Tasks

1. Load CSV into DataFrame.
2. Display first/last 5 rows.
3. Check null values & fill missing.
4. Describe dataset (mean, median, mode).
5. Sort dataset by column.
6. Group by and aggregate data.
7. Line chart (Matplotlib).
8. Bar chart (Matplotlib).
9. Histogram (Seaborn).
10. Scatter plot (Seaborn).

References

- Programiz – Pandas (<https://www.programiz.com/python-programming/pandas>)
- GeeksforGeeks – Pandas Tutorial (<https://www.geeksforgeeks.org/python-pandas-dataframe/>)
- JavaTpoint – Pandas Tutorial (<https://www.javatpoint.com/python-pandas>)
- W3Schools – Pandas (<https://www.w3schools.com/python/pandas/default.asp>)
- YouTube: [Pandas & Data Analysis Project](#)

❖ Day 6: Final Python Mini-Project (ML / IoT / Automation)

Pick one:

- **ML:** Iris Flower Classifier (Scikit-Learn).
- **IoT:** Sensor Data Logger with CSV + Matplotlib.
- **Automation:** Web Scraper for Quotes / Weather.

Mini-Tasks

1. ML: Load dataset (CSV).
2. ML: Train/test split.
3. ML: Train Logistic Regression.

4. ML: Predict & evaluate.
5. IoT: Simulate sensor data logging in CSV.
6. IoT: Plot real-time data with Matplotlib.
7. Automation: Fetch webpage using requests.
8. Automation: Parse HTML using BeautifulSoup.
9. Save scraped data into CSV.
10. Visualize collected data.

References

- Programiz – ML & Projects (<https://www.programiz.com/python-programming/project>)
- GeeksforGeeks – Final Year Projects (<https://www.geeksforgeeks.org/python-projects-beginner-to-advanced/>)
- JavaTpoint – Python Projects (<https://www.javatpoint.com/python-projects>)
- W3Schools – Python Project Ideas (https://www.w3schools.com/python/python_examples.asp)
- YouTube: [Python Final Project Ideas](#)